

**SIGNATURE VERIFICATION SYSTEM WITH FEW-SHOT LEARNING  
AND TRIPLET LOSS**

By

Jimmy Ding Jia Kang

A PROPOSAL

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

**BACHELOR OF COMPUTER SCIENCE (HONOURS)**

Faculty of Information and Communication Technology

(Kampar Campus)

OCTOBER 2025

# **ABSTRACT**

Handwritten signature remains a widely used behavioural biometric, supporting the importance of a robust offline signature verification. However, offline signature verification remains difficult due to high intra-class variability, high inter-class similarity, and a lack of real-world training dataset. This project proposes a Convolutional Neural Network (CNN)-based model enhanced with selective contrastive triplet loss to address these limitations. The model will be trained and evaluated against benchmark signature offline signature datasets, with preprocessing steps applied. The proposed methodology aims to improve computational efficiency and accuracy by focusing on negative-anchor pairs rather than positive-anchor pairs. The proposed model is anticipated have better performance than conventional Convolutional Neural Network and Siamese network approaches.

Area of Study: Deep Learning, Offline Signature Verification

Keywords: Signature verification, Deep metric learning, Convolutional Neural Network, Triplet loss, Selective Contrastive Triplet Loss, Biometric

# TABLE OF CONTENTS

|                                                                                                       |            |
|-------------------------------------------------------------------------------------------------------|------------|
| <b>TITLE PAGE</b>                                                                                     | <b>i</b>   |
| <b>ABSTRACT</b>                                                                                       | <b>ii</b>  |
| <b>TABLE OF CONTENTS</b>                                                                              | <b>iii</b> |
| <b>LIST OF FIGURES</b>                                                                                | <b>iv</b>  |
| <b>LIST OF TABLES</b>                                                                                 | <b>v</b>   |
| <b>LIST OF ABBREVIATIONS</b>                                                                          | <b>vi</b>  |
| <br>                                                                                                  |            |
| <b>CHAPTER 1 PROJECT BACKGROUND</b>                                                                   | <b>1</b>   |
| 1.1 Introduction                                                                                      | 1          |
| 1.2 Problem Statement                                                                                 | 2          |
| 1.3 Motivation                                                                                        | 2          |
| 1.4 Contributions                                                                                     | 3          |
| <br>                                                                                                  |            |
| <b>CHAPTER 2 LITERATURE REVIEW</b>                                                                    | <b>4</b>   |
| 2.1 Existing Implementations of Signature Verification Systems                                        | 4          |
| 2.1.1 Survey paper on Deep Learning Signature Verification Systems                                    | 4          |
| 2.1.2 Offline Signature Recognition and Forgery Detection using Deep Learning                         | 5          |
| 2.1.3 Convolutional Neural Network (CNN) Based Offline Signature Verification Application             | 6          |
| 2.1.4 Enhancing Signature Verification Using Triplet Siamese Similarity Networks in Digital Documents | 6          |
| 2.1.5 Learning Metric Features for Writer-Independent Signature Verification using Dual Triplet Loss  | 8          |
| 2.1.7 A Two-Stage Siamese Network Model for Offline Handwritten Signature Verification                | 9          |
| 2.1.8 Hard Negative Examples are Hard, but Useful                                                     | 10         |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| 2.1.9 Offline Signature Verification Using Region-Based<br>Deep Learning Network | 12        |
| 2.2 Limitation of Previous Studies                                               | 13        |
| 2.3 Proposed Solutions                                                           | 14        |
| <b>CHAPTER 3 PROJECT SCOPE AND OBJECTIVES</b>                                    | <b>16</b> |
| 3.1 Project Scope                                                                | 16        |
| 3.2 Project Objectives                                                           | 17        |
| <b>CHAPTER 4 METHODS/TECHNOLOGIES INVOLVED</b>                                   | <b>18</b> |
| 4.1 Development Methods                                                          | 18        |
| 4.1.1 CRISP-DM                                                                   | 18        |
| 4.1.2 Waterfall Methodology                                                      | 20        |
| 4.2 System Design                                                                | 21        |
| 4.2.1 Hardware                                                                   | 21        |
| 4.2.2 Software                                                                   | 22        |
| 4.2.3 System Workflow                                                            | 22        |
| 4.2.3.1 Data Acquisition and Preparation                                         | 24        |
| 4.2.3.2 Feature Extraction                                                       | 24        |
| 4.2.3.3 Evaluation Strategy                                                      | 24        |
| 4.2.3.4 Training Loop                                                            | 24        |
| <b>REFERENCES</b>                                                                | <b>26</b> |

## LIST OF FIGURES

| Figure Number    | Title                | Page |
|------------------|----------------------|------|
| Figure 4.1.1     | CRISP-DM [14]        | 18   |
| Figure 4.2.3.1   | System Block Diagram | 23   |
| Figure 4.2.3.4.1 | Training Loop        | 25   |

## LIST OF TABLES

| Table Number  | Title                                     | Page |
|---------------|-------------------------------------------|------|
| Table 2.1.1.1 | Offline Signature Datasets Comparison [2] | 4    |
| Table 4.1     | Specifications of Laptop                  | 21   |

## LIST OF ABBREVIATIONS

|             |                                      |
|-------------|--------------------------------------|
| <i>HoG</i>  | Histogram of Orientated Gradients    |
| <i>LBP</i>  | Local Binary Patterns                |
| <i>AI</i>   | Artificial Intelligence              |
| <i>FRR</i>  | False Rejection Rate                 |
| <i>FAR</i>  | False Acceptance Rate                |
| <i>ROC</i>  | Receiver Operating Characteristic    |
| <i>EER</i>  | Equal Error Rate                     |
| <i>CNN</i>  | Convolutional Neural Network         |
| <i>SURF</i> | Speeded-Up Robust Features           |
| <i>ROI</i>  | Region of Interest                   |
| <i>WD</i>   | Writer-Dependent                     |
| <i>WI</i>   | Writer-Independent                   |
| <i>SSN</i>  | Siamese Similarity Network           |
| <i>tSSN</i> | Triplet Siamese Similarity Network   |
| <i>SVM</i>  | Support Vector Machine               |
| <i>LR</i>   | Logistic Regression                  |
| <i>SGD</i>  | Stochastic Gradient Descent          |
| <i>SCT</i>  | Selectively Contrastive Triplet Loss |
| <i>HN</i>   | Hard Negative                        |
| <i>SHN</i>  | Semi-Hard Negative                   |

|                  |                                                 |
|------------------|-------------------------------------------------|
| <i>MSDN</i>      | Mutual Signature DenseNet                       |
| <i>SE-Blocks</i> | Squeeze-and-Excitation blocks                   |
| <i>SPP</i>       | Spatial Pyramid Pooling                         |
| <i>PDPA</i>      | Personal Data Protection Act                    |
| CRISP-DM         | Cross-Industry Standard Process for Data Mining |



## CHAPTER 1

### Project Background

In this chapter, the project background, motivation, and contributions, as well as what my project has to offer.

#### 1.1 Introduction

The field of biometric authentication has seen significant advancements, with methods such as fingerprint, facial, and iris recognition becoming ubiquitous. As the world becomes more digitalised – from the introduction of touch-screen devices to e-wallets – certain authentication technologies and tools that seem out-of-date remain relevant. One of which is the use of handwritten signatures. Handwritten signature verification remains a critical area, particularly for documents that require a physical signature to be legally binding. For instance, in Malaysia, according to the Electronic Commerce Act 2006, digital signatures cannot be used in documents such as the power of attorney, wills, codicils, trusts, and negotiable instruments [1]. Although verifying handwritten signatures plays a crucial role in authentication, the process remains largely manual, resulting in significant time consumption and increased vulnerability to human error. This challenge has led to the development of automated signature verification systems. Automated signature verification systems streamline the process by allowing only signatures that fall within a certain threshold [3], drastically reducing the volume of documents requiring manual review. Yet these systems struggle with the natural variations and inconsistencies inherent in human handwriting [2].

Signature verification systems are divided into two categories: online and offline systems. In online systems, signatures are obtained via an electronic input device, collecting not only the final image but also the dynamic temporal information – acceleration, speed, pressure, and pen angle. On the other hand, offline or static signatures are obtained by scanning the document. Due to this method of collection, the dynamic characteristics are lost, making verification more challenging. Earlier approaches to automatic signature verification primarily rely on geometric and

statistical methods such as Histogram of Orientated Gradients (HoG) and Local Binary Patterns (LBP). However, recent research trends are leaning towards deep-learning techniques for enhanced accuracy and adaptability [3].

The shift in techniques introduced a new set of problems to be addressed, such as the lack of real-world data for each class, high computational costs, and high error rates. This project aims to address these challenges by developing robust, cost-effective systems that companies and organisations can integrate into their verification pipeline.

### **1.2 Problem Statement**

Balancing computational efficiency with verification accuracy in the verification of offline signatures using deep learning models remains a significant challenge. This arises from the need for large datasets to train deep learning models; without sufficient training or fine-tuning data, the model would perform poorly [2].

Moreover, human experts are still relied upon to distinguish between genuine and forged signatures; however, they are susceptible to exhaustion when faced with high volumes of documents, leading to a higher rate of errors. Such errors in verification can lead to dire consequences, particularly in financial, legal, and administrative contexts where a reliable and efficient verification system is critical.

Furthermore, current implementations of offline signature verification still struggle with intra-class variability and filtering skilled forgeries. These challenges highlight the need for improved verification that can maintain high accuracy while operating efficiently in real-world environments.

### **1.3 Motivation**

This project is driven by the need to enhance existing implementations by exploring alternative deep learning techniques for offline signature verification, aiming to broaden their capabilities while preserving high processing speed and operational efficiency. Addressing the gap – difficulty in balancing computational efficiency and verification accuracy – motivates the exploration of alternative architectures and optimisation strategies.

The motivation also stems from the continuing importance of handwritten signatures in our modern society. Despite the emergence of digital authentication methods, handwritten signatures remain legally and culturally significant. A reliable and efficient verification system, therefore, plays a crucial role in reducing fraud, safeguarding financial and legal assets, and maintaining trust in institutional processes.

Finally, this project aligns with the goal of integrating AI as a collaborative tool rather than a replacement for human expertise. Professionals tasked with verifying signatures remain indispensable, but they are prone to fatigue and error when handling large volumes of documents. By providing an assistive tool that reduces repetitive workload and highlights highly suspicious cases, verification can be carried out more efficiently and reliably.

### **1.4 Contributions**

This project will explore alternative deep learning techniques and architectures designed to achieve an optimal balance between verification accuracy and computational efficiency. By focusing on both robustness and speed, the system will address the challenge of balancing performance and efficiency.

Banks, legal firms, governmental bodies, non-governmental bodies, or any systems, organisations, or companies that actively use handwritten signatures would benefit from this system. The signature verification system will feature a flexible, modular design, enabling seamless integration into existing pipelines without disrupting current workflows. To maintain optimal response time and throughput, the system will be designed for speed and robustness, ensuring a seamless flow between one module. This system will also minimise wait time and possible hiccups during transitions between modules. This is necessary to ensure practical usability in real-world applications where large volumes of signatures must be verified quickly and accurately.

Finally, by demonstrating a proof-of-concept system that combines speed, robustness, and modularity, this project serves as a potential foundation for further research into efficient biometric verification methods.

## CHAPTER 2

### Literature Review

#### 2.1 Existing Implementations of Signature Verification Systems

##### 2.1.1 Survey paper on Deep Learning Signature Verification Systems

In general, signature verification involves four main stages: data acquisition or digitisation of handwritten signatures, preprocessing of the acquired signatures, feature extraction for the statistical approach or structural representation for the structural approach, and classification of the provided signature [2, 3]. Depending on the types of signatures – offline or online signatures – different methods and datasets are employed, due to the difference in their nature. The most used offline signature datasets are shown in Table 2.1.1 [2].

Table 2.1.1.1: Offline Signature Datasets Comparison [2]

| Dataset name          | Language             | Number of Signers | Genuine | Forged | Total Signatures |
|-----------------------|----------------------|-------------------|---------|--------|------------------|
| <b>CEDAR</b>          | Multiple backgrounds | 55                | 24      | 24     | 2640             |
| <b>MCYT-75</b>        | Spanish              | 75                | 15      | 15     | 2250             |
| <b>GPDS-syntheses</b> | Computer-generated   |                   | 24      | 30     | 4000             |
| <b>SigComp`11</b>     | Dutch                | 64                |         |        | 1932             |
| <b>SigComp`11</b>     | Chinese              | 20                |         |        | 1177             |
| <b>SigWiComp`13</b>   | Japanese             | 30                | 42      | 36     | 2340             |

The existing implements are further distinguished by the method in which the signatures are represented in the system – statistical or structural. Structural representations, such as graph-based and contour-based models, generally achieve better verification accuracy. However, these approaches typically require greater computational resources and larger storage and result in slower response times compared to feature-embedding methods [3]. Statistical representation revolves around extracting features from the signature either on a local or global scale. The local scale

focuses on analysing sub-regions of the signature image, capturing independent details such as stroke patterns, textures, or localized distortions, whereas the global scale analyses the entire signature image to produce a single feature vector [3].

These implementations can be categorised as either writer-dependent or writer-independent, with the latter being more prevalent. In a writer-dependent approach, the system implements a separate model for each signer, resulting in higher storage requirements and computational power demands. In contrast, writer-independent approaches train a single model on the entire dataset, offering greater flexibility [2]. Since signature verification is a classification problem, its performance can be evaluated using the false rejection rate (Type I error) and false acceptance rate (Type II error). These metrics can be further used to create a receiver operating characteristic (ROC) curve and determine the equal error rate (EER) [3].

$$FRR = 1 - \frac{TP}{\text{number of genuine signatures}} \quad (1)$$

$$FAR = \frac{FP}{\text{number of forged signatures}} \quad (2)$$

The utilisation of deep learning models in signature verification generally takes the form of deep metric learning. Consequently, the choice of distance metrics holds a lot of weight in determining verification accuracy. Euclidean distance and Manhattan distance are amongst the more popular choices due to their simplicity and compatibility with compact or isolated clusters. However, both metrics are sensitive to outliers [2].

### 2.1.2 Offline Signature Recognition and Forgery Detection using Deep Learning

[4] demonstrated the efficacy of a system that combines both deep learning and classical feature-based approaches. The authors proposed a hybrid approach that employs a Convolutional Neural Network (CNN) for signature verification and a Crest-Trough algorithm for forgery detection, with Speeded-Up Robust Features (SURF) and the Harris corner detection algorithm applied as post-checks. Their preprocessing steps include noise removal, scaling, centralisation, and rotation normalisation to isolate and

align the signature region of interest (ROI) before feature extraction and CNN processing. The authors report strong empirical performance, achieving 94% accuracy in signature recognition and forgery detection rates ranging from approximately 85% to 89%.

However, this study is limited to a CNN combined with a feature-engineering pipeline and does not explore metric-learning approaches, Siamese networks, triplet embedding strategies, or robustness experiments against image distortions. As such, it serves as a useful baseline for future research.

### **2.1.3 Convolutional Neural Network (CNN) Based Offline Signature Verification Application**

CNNs are well-suited for signature verification tasks because they can automatically extract discriminative local and global spatial features, reducing reliance on handcrafted feature engineering [5]. The authors proposed an ad hoc CNN-based approach for offline signature verification, evaluating both Writer-Dependent (WD), where models are trained separately for each signer, and Writer-Independent (WI), where a single model is trained across multiple signers. The authors trained their models on the GPDS Synthetic Signature dataset, which contains 4,000 signers with both genuine and forged signatures. The CNN architecture consisted of five convolutional layers, two pooling layers, three dense layers, and dropout, with greyscale images resized to 210x300 as input. Results showed that the WD models achieved 75% accuracy, while the WI models achieved 62.5% accuracy. The authors noted that while their ad hoc CNN performed reasonably well, the absence of complementary feature extraction methods limited performance, suggesting hybrid approaches may yield improvements.

### **2.1.4 Enhancing Signature Verification Using Triplet Siamese Similarity Networks in Digital Documents**

[6] proposed their implementation of a triplet Siamese similarity network (tSSN) to authenticate signatures. They experimented with multiple distance metrics – Euclidean, Manhattan, and Minkowski distances. Their method preprocesses signatures via

bounding-box ROI extraction, greyscale conversion, and normalisation before feeding them into a lightweight CNN backbone. By training on triplets (anchor, positive, negative), the model learns an embedding space where genuine signatures cluster closely, while forged signatures are pushed apart. A two-layer similarity network then performs the final classification. The authors reported that their tSSN architecture outperformed both a standalone CNN and a traditional Siamese network, highlighting the effectiveness of triplet loss for distinguishing highly similar signatures.

The authors evaluated their model on several benchmark datasets: BHSig260, SigComp2011, 4NSigComp2010, 4NSigComp2012, CEDAR, MCYT, and GDPS with FAR and FRR as evaluation metrics. Across these datasets, the model consistently outperformed conventional CNNs and standard Siamese networks, achieving higher accuracy scores. For example, on 4NSigComp2010, tSSN reported an F1-score of 90.1%, compared to 60.5% from CNN and 75.2% from Siamese-based CNN. However, robustness testing revealed a major limitation: the system was highly sensitive to geometric transformations such as rotation, scaling, and flipping, with performance dropping to the 55–65% range.

Additionally, in comparison to other loss functions, the triplet loss function outperformed contrastive loss and binary cross-entropy loss. For example, at SigComp2011, the accuracy recorded for the triplet loss function was 92.2%, while contrastive loss recorded 76.1% and binary cross-entropy recorded 70.2%. The authors' implementation also surpassed other pre-trained models – VGG16, ResNet, MobileNet and EfficientNet – consistently across multiple datasets. Outside the domain of deep learning models, tSSN also consistently outperformed machine learning algorithms such as Support Vector Machine (SVM), Logistic Regression (LR), and Gradient Boosting (XGBoost) Classifier.

Finally, the Manhattan distance metric was found to be a better choice compared to the Euclidean distance and Minkowski distance metrics. However, the authors noticed that removing any of the components from the model would cause a significant reduction in performance across the datasets.

### 2.1.5 Learning Metric Features for Writer-Independent Signature Verification using Dual Triplet Loss

In 2021, Q. Wan and Q. Zou introduced the use of dual triplet loss for a writer-independent signature verification solution [7]. The core principle of triplet loss remains – minimising the distance between reference or anchor signatures and genuine signatures while widening the distance between reference and forged signatures. Their novelty lies in the system’s ability to classify skilled forgeries from random forgeries; skilled forgeries should be closer to the reference signature than random forgeries. They employed a Convolutional Neural Network (CNN) with a Euclidean distance metric, computing the loss function, accordingly, as presented in equation (3).

$$L = \Sigma_{\tau} [\lambda \times L_{skilled} + (1 - \lambda) \times L_{random}] \quad (3)$$

To reduce the computational cost, they have employed a batch hard strategy and soft margin, removing any invalid or easily satisfied triplets during training. If the author had not used a soft margin, the loss would be zero when the desired distance constraint is met. The problem with this is that it leads to zero gradients, meaning the model stops learning to further optimise the feature embeddings. Implementing a soft margin loss will penalise the model continuously, albeit at a reduced rate, even after the primary margin has been satisfied. This encourages the model to keep learning and to minimise the distance between genuine and reference signatures and maximise the distance between forged and reference signatures.

For evaluation, the authors pre-trained their model on a large synthetic dataset – the GPDS dataset – and then used a transfer learning approach to fine-tune it on real-world datasets – the MCYT and CEDAR datasets – demonstrating its effectiveness and generalisability. They also ensured a fair comparison by evaluating their model against a state-of-the-art baseline under the same experimental conditions. The authors utilised several standard metrics for signature verification, including the Area Under the Curve (AUC), False Rejection Rate (FRR), False Acceptance Rate (FAR), and the Equal Error Rate (EER). Their results demonstrated that their model surpassed the former state-of-the-art SigNet-F model on the GPDS Synthetic dataset, achieving a best EER of 7.34%



with 12 reference samples. The pre-trained model was highly competitive on the CEDAR dataset, and after fine-tuning, it performed exceptionally well, achieving a near-perfect 0.0% EER. While results on the MCYT dataset were less impressive, the model still performed competitively, reporting a best EER of 12.83%. The authors speculate that the performance difference might be due to the dataset distributions. An additional benefit of their approach is a significant reduction in computational overhead compared to traditional writer-dependent classifiers like Support Vector Machines (SVMs).

While testing the SigCNN model on the Chinese dataset CSIG-WHU, the authors found that it showed better generalisation abilities compared to the SigNet-F model. However, both models performed worse on this Chinese dataset than they did on the CEDAR and MCYT datasets, and fine-tuning using a few users barely made any difference. They conjectured that the difference in features between character-based and alphabet-based signatures could be the reason for the drop in performance. Additionally, the benchmark datasets used for training are often highly controlled and do not represent the natural variations found in real-world signatures, such as differences in pen thickness or slight changes in a person's writing style.

### **2.1.7 A Two-Stage Siamese Network Model for Offline Handwritten Signature Verification**

Xiao and Ding [8] addressed the challenges of class imbalance and weak feature representation in offline handwritten signature verification by proposing a two-stage Siamese neural network (SNN). Unlike traditional CNN-based pairwise models, their architecture included an additional image-enhancement branch designed to amplify discriminative writer-style features before comparison. To address the strong class imbalance between positive and negative pairs, the authors adopted focal loss in place of standard cross-entropy. This approach enhances the model's sensitivity to hard-to-classify samples while mitigating any possible dominance of the majority class.

The model consisted of two parallel Siamese paths. The first processed the original signature pair directly, while the second processed an enhanced version produced by a

lightweight attention-like module. Specifically, this enhancement path used global average pooling on intermediate CNN feature maps, followed by fully connected layers, reshaping, and up sampling to create a weight matrix. This matrix was multiplied with the original signature image to emphasise salient regions. Both paths were trained using focal loss, and their respective losses were combined via a weighting factor, enabling the model to jointly optimise raw and enhanced representations for improved discriminative capability. The backbone CNN comprised four convolutional blocks, each followed by batch normalisation and max pooling. The system was implemented in PyTorch and optimised using stochastic gradient descent (SGD).

The approach was evaluated in a writer-independent setting on CEDAR, BHSig-Bengali, BHSig-Hindi, and a Chinese forensic signature dataset. Results demonstrated superior accuracy compared to baselines such as SigNet, morphology features, and ensemble learning methods, achieving 95.7% on CEDAR and over 90% on BHSig-Bengali. Ablation studies confirmed the advantage of combining the enhancement branch with focal loss, as both single-stage SNN and image-enhanced SNN underperformed relative to the proposed two-stage design.

The key contributions of this work include the introduction of a two-stage Siamese network with feature enhancement, the application of focal loss to address class imbalance in the HSV dataset, and an evaluation on a novel real-world Chinese signature dataset.

### **2.1.8 Hard Negative Examples are Hard, but Useful**

Signature verification systems commonly use a triplet loss function, whereby the reference signature serves as the anchor image, the original signature becomes the positive image, and the forged signature becomes the negative image. However, optimising triplet loss in deep metric learning is found to be challenging [9]; choosing the hardest negative examples for deep metric learning causes bad local minima in the early phases of the optimisation. To prevent this, authors have developed semi-hard triplet mining, whereby the negative examples are almost as close to the anchor as positive examples.

[9] found that if the gradient of the loss function does not account for the normalisation of feature vectors onto a hypersphere during backpropagation, much of the gradient information gets discarded when the features are re-normalised, especially for triplets with closely located points. Additionally, when training the network on images from different classes that have already been mapped to similar feature representations, the loss gradient might mistakenly draw these points closer together instead of pushing them apart, which contradicts the intended objective.

To address these challenges, the authors proposed Selectively Contrastive Triplet Loss (SCT), which discards anchor-positive pairs and focuses exclusively on anchor-negative pairs. By applying a contrastive loss that penalises high similarity between dissimilar samples, SCT effectively enhances the ability to distinguish closely resembling but semantically unrelated inputs.

$$L_{SC}(S_{ap}, S_{an}) = \{\lambda S_{an} L(S_{ap}, S_{an}) \frac{if S_{an} > S_{ap}}{others} \#(4)$$

The authors compared their approach to triplet loss implemented with the Hard Negative (HN) mining strategy and the Semi-Hard Negative (SHN) mining strategy and evaluated their strategy with the CUB200 (CUB), CAR196 (CAR), Stanford Online Products (SOP), In-shop Cloth (In-shop), and Hotels-50K (Hotel) datasets. The images are augmented with a standard data augmentation scheme and normalised with the channel means and standard deviations. All models were trained using stochastic gradient descent (SGD) without applying any momentum. Compared to HN and SHN strategies, SCT demonstrates superior ability to avoid convergence to poor local minima. By exclusively minimising anchor-negative similarity, rather than enforcing distance between anchor-positive pairs, the model encourages a more dispersed embedding space, which helps mitigate overfitting and facilitates more effective feature transfer across domains.

### 2.1.9 Offline Signature Verification Using Region-Based Deep Learning Network

Li Liu, Linlin Huang, Fei Yin, and Youbin Chen [10] proposed an approach that segments each signature into multiple overlapping regions, extracts local feature representations, and computes similarity scores using a region-based metric-learning model. For the final decision, the authors fused the similarity scores from individual regions to produce a final classification.

The authors pre-processed signature images by adjusting the angle of the signature, removing the background, and normalising the foreground greyscale values and image dimensions. Furthermore, if the signature images are in colour, the authors ensured their conversion into greyscale using Otsu's binarisation method, which simultaneously removed the background image by segmenting the foreground signature from its surroundings. Subsequently, the signature images underwent size normalisation to standardise the size and location of the signatures. This is achieved using the moment normalisation method.

The novelty proposed by the authors lies in their feature extraction section, whereby they designed a Mutual Signature DenseNet (MSDN), which integrates both high- and low-level features, taking the local region as input to produce more distinctive feature representations. The proposed MSDN used DenseNet-36 as the backbone of the model. Their model comprised two CNN branches with a similar structure and parameters. They used squeeze-and-excitation blocks (SE-Blocks) and spatial pyramid pooling (SPP) to fully utilise the features of two signature images. While the inter-signature similarity is calculated by combining values of multiple local regions, a Siamese network with MSDN is used to calculate the similarity metric for a local region.

The authors experimented on multiple benchmark datasets – CEDAR, GPDS Synthetic, and CHnSig – using False Reject Rate (FRR), False Accept Rate (FAR), and Equal Error Rate (EER) as evaluation metrics to assess the model. The authors implemented the model using both the writer-dependent (WD) and writer-independent (WI) approaches.

In their writer-independent experimentations, the authors first apply their model to each region before exploring different combinations of local regions. The authors found that the combination of the centre region with edge regions performed consistently better than other combinations involving more regions. Beyond experimenting with various combinations of local regions, the configuration of MSDN plays a significant role. Incorporating SE-Blocks enhanced performance on larger datasets such as GPDS and ChnSig, while relying solely on multi-level feature representation risked overfitting. The combination of SE-Blocks and multi-level features yielded the best results on larger datasets, but not on simpler datasets like CEDAR, as a smaller sample size generally requires a simple model, lest it risk overfitting.

In comparison to the writer-independent approach, the writer-dependent approach yielded lower error rates across the dataset. Even with one reference per writer, WD performed better, and increasing the number of references further reduces errors, with 12 references yielding the lowest rates.

Finally, in WI validation, the proposed method outperformed existing approaches on both the CEDAR and GPDS datasets. In the WD setting, it also outperformed previous methods, despite relying solely on threshold adjustment using reference samples from test writers. No network model fine-tuning was done, in contrast to other methods that retrain their models directly on the test writers' data.

## **2.2 Limitation of Previous Studies**

Most of the existing studies agree that deep learning approaches generally struggle with the lack of diverse training data. For example, authors in [2] pointed out that real-world scenarios often do not provide large volumes of clean signature images, causing a lack of training data. Additionally, handwritten signatures are widely accepted as a form of biometric data. This information is relevant because most laws around the world consider this to be sensitive personal data and establish the need for consent. For example, in 2024, an amendment to the Personal Data Protection Act (PDPA) was made, whereby the definition of sensitive personal data has been expanded to explicitly include biometric data [11]. Moreover, there exists an imbalance between the

distribution of genuine and forged signature images, which can affect the performance of machine learning models [8]; in real life, the number of forged signatures far outnumbers the count of genuine signatures, limiting the amount of data to train the model. Additionally, there is substantial intra-class variation in signatures; according to [12], the probability of two signatures being identical is extremely low. Such variability poses challenges, such as higher rates of both false positives and false negatives for deep learning-based verification systems. According to [13], deep learning models struggle with intra-class variations that may be obvious to human beings; however, performance can be improved by reducing intra-class variance.

Additionally, offline signatures inherently lose most of the dynamic information – writing speed, pen pressure, tilt, and stroke order and direction [2]. This is due to the method in which offline signature images are obtained; they are obtained by scanning documents. The loss of information limits the amount of information researchers can work on, worsening the challenge of limited data.

In terms of optimisation, works that involve triplet loss face challenges in optimising with the hardest negative examples – skilled forgeries – often lead to bad local minima in early phases [9]. The authors also pointed out that poor triplet loss optimisation may pull the negative points closer to the positive and anchor points instead of separating them.

### **2.3 Proposed Solutions**

The proposed system utilises a triplet loss approach – Selectively Contrastive Triplet Loss (SCT) – proposed by [9], which has yet to be explored in the domain of offline signature verification. While conventional triplet loss and standard contrastive loss have been applied in this field, SCT has not been explored for signature verification tasks.

The key motivation for using SCT lies in its ability to improve both computational efficiency and discriminative power. By focusing only on widening the distance between the negatives and the anchors, faster convergence and more robust

generalisation in scenarios with high intra-class variability, which is common in signature verification tasks.

The implementation will involve integrating SCT into a signature verification pipeline, embedding it within a chosen network backbone. Its performance will be benchmarked against standard metric learning approaches, such as classic triplet loss and contrastive loss, to evaluate improvements in efficiency, robustness, and accuracy.

Additionally, to ensure broad applicability and cultural generalisability, experiments will be conducted using multiple datasets, including CEDAR, MCYT-75, and SigComp11. These datasets provide diverse handwriting styles and cultural variations, allowing the system to be assessed under conditions that resemble practical deployment scenarios.

## CHAPTER 3

### Project Scope and Objectives

#### 3.1 Project Scope

By the end of the project, the goal is to deliver a fully functioning offline signature verification system. It will contain a web application for demonstrations in which users will insert their documents with signatures and view the corresponding metrics.

- The system can detect signatures from a document and extract them automatically.
- Implementation of a deep learning model using a few-shot learning and triplet loss to validate the extracted signatures.
- Storage of reference signatures in a secure database.
- Display of performance metrics related to the verification process.
- The system would be able to be integrated within any pipeline that requires verification of signatures.
- Publicly available signature datasets such as the CEDAR dataset will be used.

The following are outside the scope of this project:

- Verification of digitally created signatures (e.g., stylus on tablet, e-signature tools), as these require different data processing methodologies and security mechanisms.
- Mobile application development. Initial demonstration will be limited to web browsers.



### 3.2 Project Objectives

The primary objective of this system is to establish a faster, more seamless, and highly secure mechanism for offline signature verification. By intelligently filtering and prioritising signatures using a deep learning model for manual inspection, the system significantly minimises the verification workload borne by authorised personnel. Rather than replacing human involvement, the system is designed to augment and support the existing verification process, thereby enhancing operational efficiency and reducing cognitive burden.

- To tackle the manual efforts needed in offline signature verification
  - Traditional verification methods depend heavily on human analysis, which is susceptible to fatigue-induced errors and inconsistencies.
  - By filtering obvious forgeries, the system alleviates repetitive workload, allowing staff to focus only on signatures flagged as ambiguous or high-risk.
- To improve verification accuracy and consistency
  - The system leverages pattern recognition capabilities of deep learning to detect variations in signature strokes, alignment, and formation.
- To enhance security and fraud detection
  - The model will be able to identify subtle signs of forgery or tampering
- To accelerate processing time for high-volume tasks
  - Environments where hundreds or thousands of signatures are processed daily will have their efficacy improved as the approval workflows are sped up.
- To support human decision-making, not replace it
  - This tool is designed as a collaborative tool, with human verifiers to handle ambiguous cases.
  - This tool aims to combine human's intuition to make careful decisions and automation to streamline tedious verifications.

## CHAPTER 4

### Methods

The development of the offline signature verification system will follow a hybrid approach, integrating CRISP-DM and waterfall methodology. The CRISP-DM framework guides the development of the deep learning model. In parallel, the waterfall methodology provides a framework for system development, focusing on modular integration, performance optimisation, and prototype deployment.

#### 4.1 Development Methods

##### 4.1.1 CRISP-DM

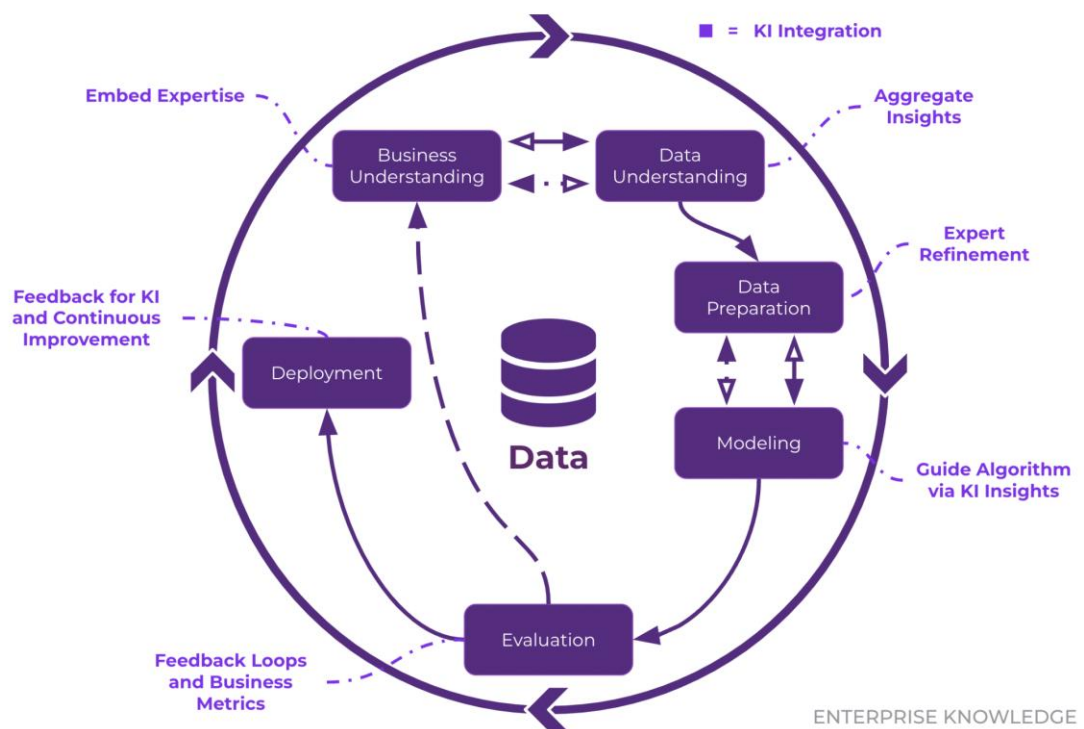


Figure 4.1.1 CRISP-DM [14]

#### 1. Business understanding

- a. Define the problem of offline signature verification and the limitations of existing approaches
- b. Establish project objects:

- i. Apply Selective Contrastive Triplet Loss (SCT) to improve performance

## **2. Data Understanding**

- a. Collect benchmark datasets
- b. Perform exploratory analysis
- c. Identify dataset limitations

## **3. Data Preparation**

- a. Preprocess images
- b. Apply data augmentation
- c. Partition datasets into training, validation, and testing splits

## **4. Modelling**

- a. Implement the proposed Selective Contrastive Triplet Loss within a deep learning backbone.
- b. Train and fine-tune models
- c. Experiment with alternative baseline losses for comparison

## **5. Evaluation**

- a. Evaluate models on metrics such as accuracy, Equal Error Rate (EER), False Acceptance Rate (FAR), False Rejection Rate (FRR), and Area Under Curve (AUC).
- b. Compare results of SCL against baseline methods to measure improvements in efficiency and discriminative power.

## **6. Deployment**

- a. Integrate the best-performing model into a modular verification pipeline.
- b. Demonstrate the prototype system in a way that it can be easily adopted into workflows

### **4.1.2 Waterfall Methodology**

The waterfall methodology is a linear and sequential approach, making it highly suitable for projects with defined requirements and deliverables. Since the objective of this project is well established, the waterfall model provides a clear structure for development. The phases are as follows:

#### **1. Requirements Analysis**

- a. Identify the needs of end-users that rely on signature verification.
- b. Define systems requirements

#### **2. System Design**

- a. Design the overall architecture of the verification pipeline
- b. Specify how the trained model will be embedded into the system

#### **3. Implementation**

- a. Integrate the model trained under CRISP-DM into the pipeline

#### **4. Testing**

- a. Conduct unit testing on individual modules and integration testing on the complete pipeline.
- b. Validate that system throughput and accuracy meet the specified requirements.
- c. Benchmark system performance in terms of response time, accuracy, and robustness under practical conditions.

#### **5. Deployment**

- a. Provide documentation and guidelines for potential adoption in banking, legal and governmental applications.

## 4.2 System Design

This chapter outlines the methodologies and technologies used in designing and implementing the signature verification system. The system will be divided into four major phases – signature detection, signature extraction, feature extraction, and signature matching/verification – each addressing specific challenges associated with real-world document analysis and authentication. The following sections describe the technical rationale, algorithms, and tools employed across each phase.

### 4.2.1 Hardware

Hardware involved in this project is a desktop computer. The computer will be used to train a deep learning model capable of correctly verifying legitimate handwritten signatures. It is also utilised for testing and refining the application during development.

Table 4.1 Specifications of Laptop

| Description      | Specifications                            |
|------------------|-------------------------------------------|
| Model            | Gigabyte Z790 UD AX                       |
| Processor        | Intel Core i7-14700K                      |
| Operating System | Windows 11                                |
| Graphic          | NVIDIA GeForce RTX 4070 SUPER 12GB GDDR6X |
| Memory           | 32GB DDR5 RAM                             |
| Storage          | 512GB SSD                                 |

### 4.2.2 Software

The system will be developed using Python, a widely adopted language for machine learning and deep learning applications, chosen for its comprehensive ecosystem of libraries that support rapid and efficient development. Visual Studio Code will serve as the primary environment for coding, debugging, and testing.

The deep learning verification model will be implemented in PyTorch, leveraging its collection of pretrained models, such as ResNet, VGG, and DenseNet, to accelerate training without compromising accuracy [15].

For demonstration purposes, Docker and PostgreSQL will be implemented to build and host the backend server. Docker will containerise the server and model, while PostgreSQL will store signature images and feature vectors. The pgvector extension will enable efficient storage, indexing, and retrieval of these vectors [16].

### 4.2.3 System Workflow

A interactive system will be developed to allow testing and demonstration of the signature verification system, as shown in figure 4.2.3.1

**Input flow:** The system accepts scanned or digital documents containing signatures.

**Phase Interactions:**

- The system detects the location of a signature in a full document.
- It extracts that signature region for analysis.
- The extracted images are processed to extract feature vectors.
- Features are matched against stored references to verify authenticity.

**Modular Architecture:** Each phase is designed as an independent module, providing flexibility to replace or upgrade components without impacting the rest of the system. For example, the signature detection component can be substituted with an alternative implementation without forcing changes to other modules.

**Performance Goal:**

- High accuracy in high intra-class variability
- Robustness to different signature styles
- Efficient processing suitable for real-time or batch verification.

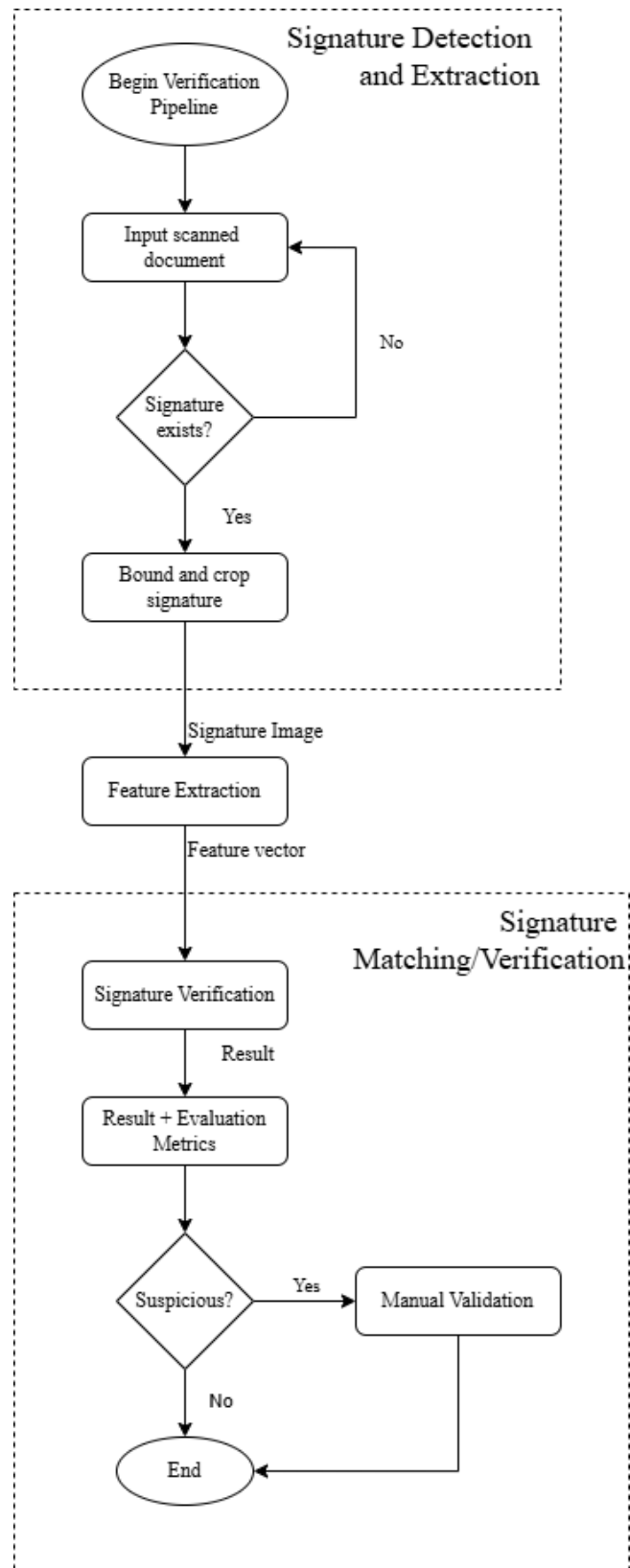


Figure 4.2.3.1: System Block Diagram

#### **4.2.3.1 Data Acquisition and Preparation**

The model will be trained using publicly available signature image datasets. To improve its capability across different languages and writing styles, datasets such as BHSig260, MCYT-100, SVC2004, ChiSig, and OHSDA will be used. All signature images will be converted to a grayscale colour scheme to improve computational efficiency. Furthermore, to better simulate real-world signatures, data augmentation techniques, including tilting, scaling, and displacement, will be applied during training.

#### **4.2.3.2 Feature Extraction**

Feature extraction would be done with a pretrained Convolutional Neural Network (CNN). Rather than training a model from scratch, utilising a pre-trained model leverages the powerful transfer learning capabilities of CNNs. By using pre-trained weights, the models already possess a strong foundation in recognizing visual patterns, which accelerates the training process on the more specialized signature data.

#### **4.2.3.3 Evaluation Strategy**

To evaluate the effectiveness of the model, a combination of qualitative visualisation – confusion matrix, 2D embedding distance graph, and 3D embedding distance graph – and quantitative metrics – Area Under the Receiver Operating Characteristic curve (AUC ROC) and Equal Error Rate (EER). In a 2D embedding distance graph, anchor nodes are shown in blue, positive nodes are shown in green, and negative nodes are shown in red. While a 2D projection has limitations in fully representing distances in higher-dimensional embedding space, this visualization offers some intuition into how the model has positioned different types of embeddings.

#### **4.2.3.4 Training Loop**

The model will be trained to learn feature representations based on relative distances in the embedding space (Figure 4.2.3.4.1). The training class incorporates early stopping, learning rate scheduling, checkpointing, and device management for efficient GPU utilisation. An online triplet mining strategy will be employed, using strategies such as batch-hard and batch semi-hard mining, combined with distance metrics including Euclidean distance, cosine similarity, and Manhattan distance. Additionally, Selective



Contrastive Triple Loss (SCT) adapted from [9] will be integrated to enhance discriminative capability.

Training will terminate once optimal performance is achieved, at which point the model will be saved for deployment. The model will be experimented with using various learning rate schedules and optimisers to identify the most effective configurations.

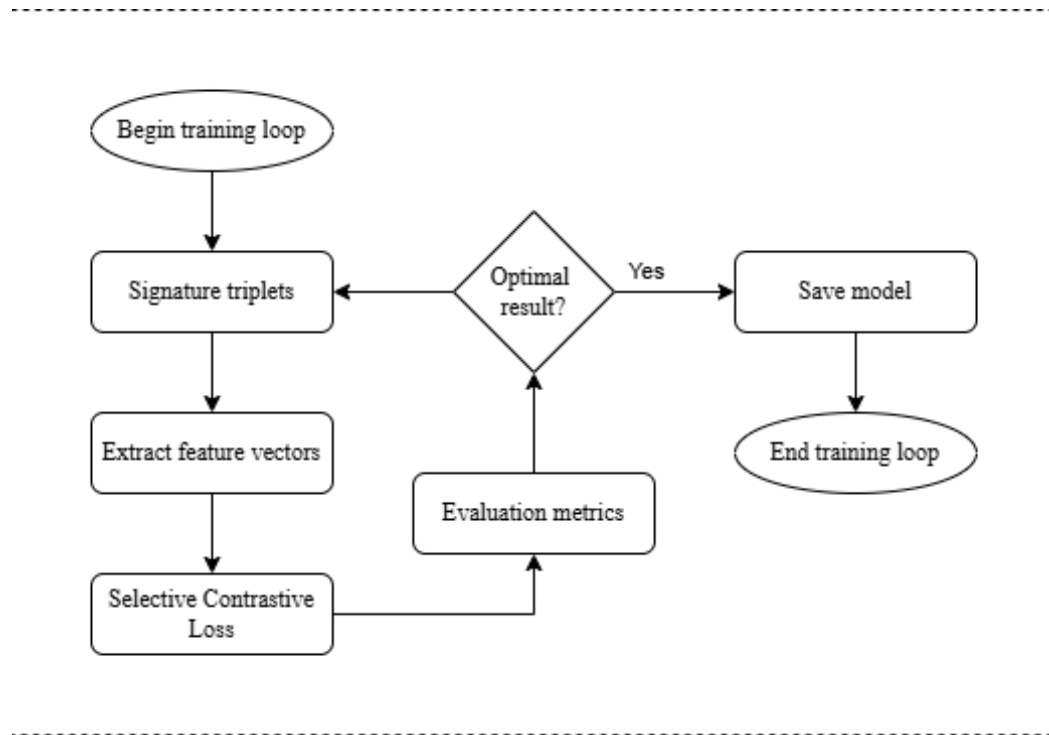


Figure 4.2.3.4.1: Training Loop

## REFERENCES

- [1] Malaysia , *Electronic Commerce Act 2006 (Act 658)*. 2006. Accessed: Aug. 24, 2025. [Online]. Available: <https://lom.agc.gov.my/ilims/upload/portal/akta/LOM/EN/Act%20658.pdf>
- [2] Z. Hashim, H. M. Ahmed, and A. H. Alkhayyat, “A Comparative Study among Handwritten Signature Verification Methods Using Machine Learning Techniques,” *Scientific Programming*, vol. 2022, pp. 1–17, Oct. 2022, doi: <https://doi.org/10.1155/2022/8170424>.
- [3] M. Stauffer, P. Maergner, A. Fischer, and K. Riesen, “A Survey of State of the Art Methods Employed in the Offline Signature Verification Process,” *Studies in systems, decision and control*, pp. 17–30, Jul. 2020, doi: [https://doi.org/10.1007/978-3-030-48332-6\\_2](https://doi.org/10.1007/978-3-030-48332-6_2).
- [4] J. Poddar, V. Parikh, and S. K. Bharti, “Offline Signature Recognition and Forgery Detection using Deep Learning,” *Procedia Computer Science*, vol. 170, pp. 610–617, Jan. 2020, doi: <https://doi.org/10.1016/j.procs.2020.03.133>.
- [5] M. Mutlu Yapici, A. Tekerek, and N. Topaloglu, “Convolutional Neural Network Based Offline Signature Verification Application,” *IEEE Xplore*, Dec. 01, 2018. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8625290> (accessed Apr. 10, 2023).
- [6] S. Tehsin, A. Hassan, F. Riaz, I. M. Nasir, N. L. Fitriyani, and M. Syafrudin, “Enhancing Signature Verification Using Triplet Siamese Similarity Networks in Digital Documents,” *Mathematics*, vol. 12, no. 17, p. 2757, Sep. 2024, doi: <https://doi.org/10.3390/math12172757>.
- [7] Q. Wan and Q. Zou, “Learning metric features for writer-independent signature verification using dual triplet loss,” 2021, pp. 3853–3859. doi: <https://doi.org/10.1109/ICPR48806.2021.9413091>.
- [8] W. Xiao and Y. Ding, “A Two-Stage Siamese Network Model for Offline Handwritten Signature Verification,” *Symmetry*, vol. 14, no. 6, p. 1216, Jun. 2022, doi: <https://doi.org/10.3390/sym14061216>.

## REFERENCES

- [9] H. Xuan, A. Stylianou, X. Liu, and R. Pless, “Hard Negative Examples are Hard, but Useful,” *Lecture notes in computer science*, pp. 126–142, Jan. 2020, doi: [https://doi.org/10.1007/978-3-030-58568-6\\_8](https://doi.org/10.1007/978-3-030-58568-6_8).
- [10] L. Liu, L. Huang, F. Yin, and Y. Chen, “Offline Signature Verification Using A Region Based Deep Metric Learning Network,” *Pattern Recognition*, vol. 118, p. 108009, May 2021, doi: <https://doi.org/10.1016/j.patcog.2021.108009>.
- [11] Department of Personal Data Protection, Malaysia, *Personal Data Protection (Amendment) Act 2024*. 2024. Accessed: Sep. 03, 2025. [Online]. Available: <https://www.pdp.gov.my/ppdpv1/en/akta/personal-data-protection-amendment-act-2024/>
- [12] M. J. Saks, “Commentary on: Srihari SN, Cha S-H, Arora H, Lee S. Individuality of handwriting. J Forensic Sci 2002; 47(4):856–72.,” *Journal of Forensic Sciences*, vol. 48, no. 4, p. 2002428, Jul. 2003, doi: <https://doi.org/10.1520/jfs2002428>.
- [13] Z. Zhang, C. Luo, H. Wu, Y. Chen, N. Wang, and C. Song, “From Individual to Whole: Reducing Intra-class Variance by Feature Aggregation,” *International journal of computer vision*, vol. 130, no. 3, pp. 800–819, Feb. 2022, doi: <https://doi.org/10.1007/s11263-021-01569-2>.
- [14] K. Schultz, “Understanding the Role of Knowledge Intelligence in the CRISP-DM Framework: A Guide for Data Science Projects - Enterprise Knowledge,” *Enterprise Knowledge*, Mar. 17, 2025. <https://enterprise-knowledge.com/understanding-the-role-of-knowledge-intelligence-in-the-crisp-dm-framework-a-guide-for-data-science-projects/>
- [15] TorchVision Contributors, “Models and pre-trained Weights – Torchvision Main Documentation,” *Pytorch.org*, 2024. <https://docs.pytorch.org/vision/main/models.html> (accessed Sep. 04, 2025).
- [16] pgvector, “pgvector,” *GitHub*, Aug. 16, 2023. <https://github.com/pgvector/pgvector> (accessed Sep. 04, 2025).